Notes for C++

Section 5 : Arrays

There are many programs in which we need to work with a large amount of very similar data. For example, if we wanted to keep data on monthly rainfall for one year we would need 12 variables – something like this:

```
float rainjan, rainfeb, rainmar, ……. , rainnov, raindec;
```

The program would get very long if we read values into these 12 variables, and then added them up, and so on. Instead we use an **array – a collection of variables <u>of the same data type</u> (e.g. all ints or all floats etc) under one name.**

Defining an array

When we create a simple variable the computer sets aside a location with that name. For example:

`float a, b;` this means that we have 2 locations like this:

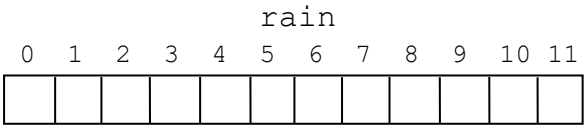


Then the instruction: `a = 4.2;` means that 4.2 is loaded into the box marked `a`.

Defining an array sets aside as many contiguous memory locations as requested, of the data type requested. For example:

`float rain[12];`

this sets up 12 locations like this:



All these locations have the same name `rain`

In the C++ language, we do not declare arrays using a variable to specify the number of boxes required. For example, do not use `float rain[num];`

Index

The name `rain` alone is not enough to work out where to store a value. For example,
`rain = 27.3;`
Where exactly would the value 27.3 be stored as there are 12 variables?

To use one of the variables in an array, we must provide the name **and** the index.
e.g. `rain[5] = 27.3;` would place 27.3 in the box marked 5.

Each variable in an array is called an **element** of the array, e.g. `rain[5]` is an element.

Note that the computer works on the first index being zero. The collection of 12 real numbers above must be declared as `float rain[12];` but the index values are numbered **0 - 11**, not 1 – 12.
The index must be an integer.

Programming with arrays often means repeatedly accessing the same array name at different positions (for assigning, manipulating, printing out etc). This means that loops (especially for loops) are very common in programs that use arrays.

Example: A program to calculate average monthly rainfall

```
#include <iostream>
using namespace std;

int indx;
float total, avg;
float rain[12];

int main() {
  total = 0.0;
  for (indx = 0; indx < 12; indx++) {
    cout << "Enter rainfall for month " << indx + 1 << ": ";
    cin >> rain[indx];
    total = total + rain[indx];
  }
  avg = total / 12.0;
  cout << "Average monthly rainfall is " << avg;
}
```

Interesting things to notice in this program:

- the array is called `rain` and has 12 ***elements*** (variables in the array)
- we read one rainfall figure at a time and load it into the correct location in the array
- we display `indx + 1` so people do not get confused by the computer counting from zero! (people understand January to be month 1 and not month 0)

Notes about arrays

- Each element of the array is a variable with a name and an ***index*** (a number)
- The index can be a variable (eg `rain[indx]`) or a value (eg `rain[5]`) or a calculation

  For example: `rain[(17 + 5) / 2] = 18.4;`

  This leads to the variable (element) `rain[11]` being assigned the value `18.4`
- Beware of going ***out of bounds*** (i.e. using an index that is too big or too small). This will cause the program to produce the wrong results but will not provide an error message.

Example: A program to calculate the maximum and minimum values in an array

```
#include <iostream>
using namespace std;

int indx, maxi, mini;
int numbers[50];

int main() {
  cout << "Enter 50 numbers\n";
  for (indx = 0; indx < 50; indx++) {
    cin >> numbers[indx];
```

```
    /* or you could load a random number into each element of the array     */
  }
  maxi = numbers[0];
  mini = numbers[0];
  for (indx = 0; indx < 50; indx++) {
    if (numbers[indx] > maxi) {
      maxi = numbers[indx];
    }
    if (numbers[indx] < mini) {
      mini = numbers[indx];
    }
  }
  cout << "Maximum is " << maxi << " and minimum is " << mini;
}
```

Interesting things to notice in this program:

- Test this program with less than 50 numbers to save typing! (e.g. maybe 8 numbers)
- `maxi` and `mini` are both initialised to the first number in the array – why is this?
- Can you put an "else" between the maximum check and the minimum check?
- Previously we wrote a program to find maximum (or minimum) without using an array. It is useful to use the array if you need to remember all the individual values for later use.

Counting arrays

We use counting arrays when we want to keep a collection of totals. For example, read in a series of NZ landline phone numbers and count how many numbers there are for each area code.

Landline numbers are 9 digits where the first two digits are the area code, e.g. 09 = Auckland and North, 04 = Wellington, 03 = South Island, etc. The first digit is always zero.

Example 1: the Massey (Auckland) landline is 094140800

Example 2: the Massey (Palmerston North) landline is 063505618

```
#include <iostream>
using namespace std;

int number, indx;
int total[10];           // 10 totals – one for each area

int main() {
  for (indx = 0; indx < 10; indx++) {
    total[indx] = 0;
  }

  cout << "Enter a NZ landline number (-1 to stop) ";
  cin >> number;

  while (number != -1) {
    indx = number / 10000000;
    total[indx]++;
    cout << "Enter a NZ landline number (-1 to stop) ";
    cin >> number;
```

```
  }

  cout << "Total numbers by area code are:\n";
  for (indx = 1; indx < 10; indx++) {
    cout << "Area code ";
    cout.fill('0');         // fill extra places with zeros
    cout.width(2);          // width of the display will be 2
    cout << right << indx;
    cout << " total numbers is " << total[indx] << endl;
  }
}
```

Interesting things to notice in this program:

- the array does not hold the data – it holds the counts associated with the data
- the index is calculated from the data
- some array elements will always be empty, e.g. index = 5 (there is no 05 area code)

Corresponding arrays

A class of students (we don't know exactly how many, but not more than 100) has just written a test (result out of 20). Each student has an ID number of 3 digits. Write a program that reads student IDs into one array and students result into a ***corresponding array***, then searches the ID array for a specific student ID, and prints out that student's result, or the fact that the student was not found.

Note: we will not include data validation (results must be less than 20). You could include this later.

Understanding the problem

We need an array of student ID numbers (an integer array of size 100)

We need an array of student test results of the same size (an integer array of size 100)

These will be used as ***corresponding arrays***, i.e. ID at index `i` corresponds to result at index `i`

For example:

the ID array:

0	1	2	3	4	5
947	281	643	458		

and so on….

the results array:

0	1	2	3	4	5
14	9	16	12		

and so on….

This means that student 947 has a test result of 14 and student 643 has a result of 16, etc.

Construct a solution to the problem

- Read in all the student ID numbers (how many are there?)
- Read in all the test results (how many are there?)
- Repeatedly read in a search ID from the user …
- and run through the ID array checking each element against the search ID…

- displaying either the corresponding result, or the fact that the student was not found

The program

```
#include <iostream>
using namespace std;

int temp, num, select, searchid, i;
int id[100], result[100];

int main() {
// section 1 : enter the student IDs
  i = 0;
  cout << "Enter the student IDs (-1 to stop)\n";
  cin >> temp;
  while (temp != -1) {
    id[i] = temp;
    i++;
    cin >> temp;
  }
  num = i;       // record how many array positions were filled
  cout << "You entered " << num << " numbers\n";

// section 2 : enter the test result for each student
  cout << "Now enter the test result for each student\n";
  for (i = 0; i < num; i++) {
    cout << "Enter the result for " << id[i] << ": ";
    cin >> result[i];
  }

// section 3 : ask for a student ID and search for a student
  cout << "Searching for a student - enter an ID ";
  cin >> searchid;
  select = -1;
  for (i = 0; i < num; i++) {  // run through the ID array
    if (searchid == id[i]) {
      select = i;   // record the id array index of the match
    }
  }
  if (select < 0) {   // if no match was found
    cout << "Student " << searchid << " not found\n";
  } else {
    cout << "Student " << id[select];
    cout << " has result " << result[select];
  }
}
```

Interesting things to notice in this program:

- we use a while loop in section 1. Why?
- we use for loops in the other sections. Why?
- we load the ID values into a temporary variable first, to avoid loading the trailing value into the array. Does this really matter?

- the search was on the id array, but the required data was from the corresponding position in the results array.
- checking an array element by element for a search value, is an algorithm called Linear Search.

When to stop searching

It is important to remember that there are always two different ways to end a search:

(a) you may find what you are looking for (usually known as "success")

(b) you may not find what you are looking for (usually known as "failure")

Note that possibility (b) is ***not*** an error. It is simply the situation in which you search for an item that is not in the array. You must make sure that your program can handle both of these situations.

We do not have to use `temp`

We could write this program without using `temp`. It would look like this:

```
// section 1 : enter the student IDs
  i = 0;
  cout << "Enter the student IDs (-1 to stop)\n";
  cin >> id[i];
  while (id[i] != -1) {
    i++;
    cin >> id[i];
  }
```

This means that we will load the trailing value into the array. However we will never look at the trailing value because it will be stored in element id[num] and we only look at elements where the index is less than num.

Searching algorithms

There are several well-known searching algorithms, i.e. different approaches to searching. We have used the linear search algorithm which runs through the data from the beginning to the end searching for a specified data item. This is not the most efficient searching algorithm (but it is easy to understand and program).

One way of making searching more efficient is to sort the data before you search through it. For example, if you have a long list of names and phone numbers, you do not usually use linear search to find the name you want. You would probably sort the names to make the searching more efficient.

Sorting

People always sorted numbers (and names) long before we used computers. Just to get the feel of how sorting works, let us work through a simple example without using a computer. Take some numbers:

```
48    13    27    62    9
```

Now sort the numbers (in your head) and then write them down in sorted order:

```
9     13    27    48    62
```

How did you sort them? We now need to develop an algorithm for sorting numbers.

An algorithm to sort numbers

Move along the numbers. Compare each number with the following number. If they are in the wrong order, swop them around. For example (**bold** indicates which number we are looking at):

48 13 27 62 9

We look at 48 and 13. They are in the wrong order so we swop them around to get:

13 **48** 27 62 9

Now we move on one position and look at 48 and 27. Again we swop them to get:

13 27 **48** 62 9

We move on one position and look at 48 and 62. There is no need to swop anything.

13 27 48 **62** 9

We move on one position and look at 62 and 9. We swop them to get:

13 27 48 9 62

We have now completed one ***pass*** of the data. A ***pass*** means that we have moved through the numbers from the beginning to the end. In fact we stopped one before the end – why?

However, we have not completed the sorting. Notice that 9 (for example) is still in the wrong position. How many passes will we need to ensure that 9 is placed in the correct position?

How to swop values

During sorting we need to swop two values. We achieve this with the following instructions:

```
int a, b, temp;

a = 3;         /* place an initial value in a */
b = 5;         /* place an initial value in b */
temp = a;      /* 3 lines needed to swop the two values */
a = b;
b = temp;
```

Note that we need an extra variable called `temp`. Why is this?

Note that the extra temporary variable needs to be of the same data type as the data being swopped.

A program to sort an array of integers

```
#include <iostream>
using namespace std;

int num, temp, pass, i;
int data[100];  /* the array can hold a maximum of 100 numbers */

int main() {
```

```
  num = 7;       /* we are working with a total of 7 numbers */
  for (i = 0; i < num; i++) {
    cout << "Enter a number : ";
    cin >> data[i];
  }
  for (pass = 0; pass < num; pass++) {
    for (i = 0; i < num - 1; i++) {
      if (data[i] > data[i+1]) {
        temp = data[i];
        data[i] = data[i+1];
        data[i+1] = temp;
      }
    }
  }
  cout << "The sorted array is ";
  for (i = 0; i < num; i++) {
    cout << data[i] << ", ";
  }
  cout << endl;
}
```

Interesting things to notice in this program:

- `int data[100]` means that the array can ***never hold more than*** 100 numbers
- `num = 7` means that there are currently only 7 numbers in the array
- `for (i = 0; i < num; i++)` will "move along the array". This happens several times.
- the ***outer loop*** (with `pass`) counts the number of passes needed to complete the sorting
- the ***inner loop*** (with `i`) moves along the array and stops one position short of the end
- we swop two ***elements*** of the array. Recall that array elements are variables in their own right.

Extra notes: this sorting algorithm is called Bubble Sort. It can be made considerably more efficient than the version that we have here. There are also many other sorting algorithms. Look for sorting algorithms on Google if you want to see more information.

Deleting and Inserting in sorted arrays

Assume that several student ID numbers have been read into an array and that the ID numbers have been sorted into order. See previous lectures on how to do this. There may be corresponding arrays but we will look only at the ID numbers. Also assume that we know how many ID numbers there are and that this value is stored in `num`.

Deleting from an array to maintain sorted order

A student leaves the class. Delete the ID number `481` and keep all the ID numbers in sorted order. We do this by searching for the index of the ID number to be deleted and then we "shuffle" the ID numbers that are greater than this ID one location to the left.

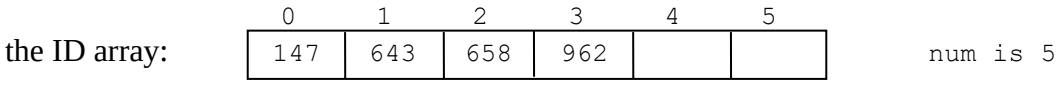
Step 1: search for the position (index) of 481 in the current array

the ID array:

0	1	2	3	4	5
147	481	643	658	962	

```
num is 5
position is 1
```

Step 2: shuffle the ID numbers left starting from position



Step 3: subtract one from num because we have removed one ID number

num is 4

```
#include <iostream>
using namespace std;

int num, removeid, position, i;
int id[100];

int main() {

// section 1: enter five student IDs in the correct order
  cout << "Enter five student IDs in the correct order\n";
  num = 5;
  for (i = 0; i < num; i++) {
    cout << "Enter ID number " << i+1 << ": ";
    cin >> id[i];
  }

// section 2: select one of these IDs to delete
  cout << "Which student ID do you want to delete? ";
  cin >> removeid;

// section 3: search for the position of the specified ID number
  position = -1;
  for (i = 0; i < num; i++) {
    if (removeid == id[i]) {
      position = i;
    }
  }
  if (position == -1) {
    cout << "ID " << removeid << " is not in the array\n";
  } else {

// section 4: shuffle array elements to the left
    for (i = position; i < num - 1; i++) {
      id[i] = id[i+1];
    }
    num--;     // one less ID number in the array
  }

// section 5: display the array
  cout << "\nHere is the full list of ID numbers\n";
  for (i = 0; i < num; i++) {
    cout << id[i] << "  ";
  }
  cout << endl;
}
```

Interesting things to notice in this program:

- there are three situations (cases) to test for – what are they? Check with walkthroughs.
- why is the shuffling loop controlled by `num - 1`, and not `num`?

Inserting into an array to maintain sorted order

A new student joins the class. Insert the ID number `502` into the correct position in the array.
We do this by searching for the correct index for inserting the new ID number and then we "shuffle" the ID numbers that are greater than the new id one location to the right starting from the end/back of the array.

Step 1: search for the correct position to insert 502 into the array

the ID array:

0	1	2	3	4	5
147	481	643	658	962	

num is 5
position is 2

Step 2: shuffle ID numbers right to make a space – start at the back of the array

the ID array:

0	1	2	3	4	5
147	481		643	658	962

num is 5

Step 3: insert the new ID number into the space in the array

the ID array:

0	1	2	3	4	5
147	481	502	643	658	962

num is 5

Step 4: add one to num because we have one more ID number

num is 6

```
#include <iostream>
using namespace std;

int num, newid, position, i;
int id[100];            //NB: the array must have extra space!

int main() {

// section 1: enter five student IDs in the correct order
  cout << "Enter five student IDs in the correct order\n";
  num = 5;
  for (i = 0; i < num; i++) {
    cout << "Enter ID number " << i+1 << ": ";
    cin >> id[i];
  }

// section 2: enter a new ID
  cout << "Now enter a new student ID ";
  cin >> newid;

// section 3: search for the correct position to insert
  i = 0;
  while (newid > id[i]) {
```

```
    i++;
    if (i == num) {
      break;
    }
  }
  position = i;

// section 4: shuffle to the right – start at the end
  for (i = num; i > position; i--) {
    id[i] = id[i-1];
  }

// section 5: insert the new ID number
  id[position] = newid;
  num++;

// section 6: display the array including the new ID number
  cout << "\nHere is the full list of ID numbers\n";
  for (i = 0; i < num; i++) {
    cout << id[i] << "  ";
  }
  cout << endl;
}
```

Interesting things to notice in this program:

- there are three situations (cases) to test for – what are they? Check with walkthroughs.
- note the break in the loop in section 3 – why do we need the break?
- step 2 of the algorithm showed a blank space in the array. Is it actually blank?
- what is the problem with using an exact array size?

Arrays of variable size -- this is NOT part of our course

In certain versions of C++ it is possible to write this:

```
int main() {
  int n;
  cout << "How big is your array? ";
  cin >> n;
  float data[n];
  // code that works with the array
}
```

This gives the impression that an array can be created where the number of elements is not defined until the program runs and the user enters a value for n. This is not the case. What appears to be an array in this program is not actually an array (because arrays always have a fixed size). It is some other data structure that uses pointers and allocates memory (etc). This will be highly inefficient compared to an array. If you want to use a different data structure then be clear about which data structure you are using and be aware of the advantages and disadvantages of the various data structures. Advanced data structures are covered in 159.201.